

PRODIGY INFOTECH • CYBER SECURITY TRACK

Cyber Security Internship Report

A one-month, task-based internship covering classical cryptography, image-level data encryption, password security auditing, and keystroke monitoring.

Shaurya Parashar

Intern — Cyber Security | 1 – 31 August 2026 | Certificate ID: PIT/AUG26/01162

Prodigy InfoTech

Prodigy InfoTech offers structured, virtual, task-based internships across Web Development, Android, Machine Learning, Data Science, and Cyber Security.

Registered under the Ministry of Micro, Small and Medium Enterprises (MSME), Government of India.

The Cyber Security track focuses on cryptography, data protection, authentication, and security-tool awareness through independent Python projects.

DOMAIN

Cyber Security

DURATION

1 Month (Aug 2026)

FORMAT

Self-paced, task-based

CERTIFICATION

MSME-registered

Objectives & Tools Used

OBJECTIVES

- Implement classical encryption via the Caesar Cipher
- Explore file-level protection with image encryption/decryption
- Build a rule-based password strength checker
- Study keystroke monitoring and its security implications
- Strengthen Python programming for security applications

TOOLS & TECH

- Python 3.11
- Visual Studio Code
- tkinter (GUI)
- datetime module
- PPM / P3 image format

Caesar Cipher Encryption Tool

Classical substitution cipher that shifts each letter by a fixed key, wrapping around the alphabet using modular arithmetic. The same function performs both encryption (+shift) and decryption (−shift).

- Shifts alphabetic characters by 'shift' positions mod 26
- Non-alphabetic characters pass through unchanged
- One reversible function handles encrypt & decrypt

```
def caesar(text, shift):  
    result = ""  
    for char in text:  
        if char.isalpha():  
            result += chr((ord(char)  
                           - ord('a') + shift) % 26  
                           + ord('a'))  
        else:  
            result += char  
    return result  
  
encrypted = caesar(msg.lower(), shift)  
decrypted = caesar(encrypted, -shift)  
  
>> Encrypted: koor  
>> Decrypted: hello
```

Image Encryption & Decryption Tool

Encrypts raw pixel data of a PPM (P3) image using an additive key, protecting media files rather than plain text.

1

Read PPM Header

Parse format, width, height, max colour value and the flat pixel list from the .ppm file.

2

Transform Pixels

encrypt: $(\text{pixel} + \text{key}) \% 256$
decrypt: $(\text{pixel} - \text{key}) \% 256$ — inverse operations restore the exact original image.

3

Write Output

Re-assemble header + transformed pixels into a new, valid P3 image file.

Image Tool — Implementation

```
def encrypt(input_file, output_file, key):
    with open(input_file, "r") as f:
        data = f.read().split()
        width, height = int(data[1]), int(data[2])
        max_value = int(data[3])
        pixels = list(map(int, data[4:]))

    for i in range(len(pixels)):
        pixels[i] = (pixels[i] + key) % 256

    with open(output_file, "w") as f:
        f.write("P3\n")
        f.write(f"{width} {height}\n")
        f.write(f"{max_value}\n")
        for p in pixels:
            f.write(str(p) + " ")
```

```
choice = input("E/D: ").upper()
input_file = input("Input image: ")
output_file = input("Output image: ")
key = int(input("Key: "))

if choice == "E":
    encrypt(input_file, output_file, key)
elif choice == "D":
    decrypt(input_file, output_file, key)
```

SAMPLE RUN

```
Enter E for Encrypt or D for Decrypt: D
Enter input image: encrypted.png
Enter output image: decrypted.png
Enter key: 50
Image decrypted!
```

Password Strength Checker

A scoring-based tool that checks a password against five common security criteria and returns a Strong / Medium / Weak verdict.

SCORING CRITERIA (+1 EACH)

- Length ≥ 8 characters
- Contains an uppercase letter
- Contains a lowercase letter
- Contains a digit
- Contains a special character



Strong

Score = 5



Medium

Score 3–4



Weak

Score < 3

Simple Keylogger (Educational)

A tkinter GUI application that captures and timestamps keystrokes while its own window is focused — built to understand keylogging mechanics and their security risks, not to capture system-wide input.

```
window.bind("<KeyPress>", key_pressed)

def key_pressed(event):
    key = event.keysym
    text_box.insert(tk.END, key + " ")

    with open("key_log.txt", "a") as file:
        time = datetime.now().strftime(
            "%H:%M:%S")
        file.write(f"{time} - {key}\n")
```

ETHICAL NOTE

This tool only listens while its own window is in focus (window.bind), unlike malicious keyloggers that hook system-wide keyboard input.

Unauthorized, system-wide keystroke capture is illegal and a serious privacy violation — this project was strictly for educational understanding.

Skills Gained

1

Cryptography Basics

Classical substitution ciphers & modular arithmetic

2

Data Protection

Encrypting binary/media data, not just plain text

3

Security Auditing

Rule-based evaluation, as in password strength scoring

4

GUI & Event Handling

tkinter event binding for real-time monitoring

5

Python Proficiency

File I/O, functions, and CLI/GUI application design

6

Security Ethics

Boundary between educational tools and malicious ones

WRAPPING UP

Conclusion

This one-month Cyber Security internship at Prodigy InfoTech gave hands-on exposure to core security concepts — from classical cryptography to media encryption, password auditing, and system-level input monitoring. Every task was implemented, tested, and verified independently, reinforcing both theory and practical Python skills.

The internship strengthened my foundation in cyber security and motivated further exploration of symmetric/asymmetric cryptography, network security, and ethical hacking.

CERTIFICATE ID

**PIT/AUG26/
01162**

ISSUED

1 Sept 2026

REMARKS

Outstanding

Thank You

Shaurya Parashar | Cyber Security Intern | Prodigy InfoTech

contact@prodigyinfotech.dev • www.prodigyinfotech.dev